

TeGy-Chatbot: AI-Powered Conversational

1. Introduction and Motivation

The proliferation of online event ticketing platforms has introduced a new class of user-experience challenges. Users frequently encounter friction when searching for events, comparing ticket prices, understanding venue logistics, or completing multi-step booking flows. Traditional FAQ pages and static help centers fail to address these needs in a personalized, context-aware manner.

This project, **TeGy-Chatbot**, addresses these challenges by providing an AI-powered conversational backend system that is integrated into an existing ticket and event booking website. The chatbot acts as an intelligent assistant capable of understanding natural-language user queries—in both Arabic and English—and responding with contextually relevant information, event recommendations, and guided booking workflows.

Conversational AI has become a preferred interaction paradigm because it reduces navigation complexity and enables users to access services using natural language rather than manually browsing multiple interface screens.

The system is implemented as a standalone backend service built with the Python FastAPI framework, designed to be consumed by a separate frontend client via RESTful API endpoints. Its architecture emphasizes fault tolerance mechanisms through multi-provider LLM integration, low-latency responses through semantic caching and a zero-LLM fast-path router, and operational resilience through distributed circuit breakers and background job processing.

1.1 Comparison with Traditional Systems

To highlight the architectural shift from legacy systems, the following table compares traditional static chatbots against the dynamic approach implemented in TeGy-Chatbot:

Feature	Traditional Chatbot	TeGy-Chatbot
Response Generation	Static FAQ & Decision Trees	Dynamic LLM Generation
AI Integration	Single AI Provider (if any)	Multi-Provider Architecture
Response Caching	No Cache / Exact Match	Vector-based Semantic Cache
Action Execution	No Tool Calling	Autonomous Tool Calling

Resilience	No Failover Mechanism	Automatic Failover & Circuit Breakers
Deployment	Monolithic	Cloud-native Containers
Scalability	Vertical	Horizontal

2. Project Objectives

The primary objectives of this project are:

1. **Event Discovery:** Enable users to find events matching their interests through natural language queries, powered by semantic search and database retrieval (app/ai/tools/event_tools.py).
2. **Intelligent Q&A:** Answer user questions about event dates, ticket prices, venues, and booking status using LLM-generated responses grounded in real database records.
3. **Booking Management:** Guide users step-by-step through ticket booking and allow retrieval or cancellation of existing bookings (app/ai/tools/ticket_tools.py, app/ai/tools/order_tools.py).
4. **Multi-Language Support:** Handle conversations in both Arabic and English, with Arabic-first branded responses for social interactions (app/ai/fast_path_router.py).
5. **Production-Oriented Reliability:** Ensure continuous availability through multi-provider LLM failover, distributed locking, idempotency guarantees, and graceful degradation when external services are unavailable.
6. **Extensible Architecture:** Provide an extensible backend architecture that can be integrated with different event booking platforms.

3. System Overview

TeGy-Chatbot is a modular, layered backend application. At the time of writing, the audited version of the repository consists of over sixty source files organized across 11 primary packages. The system integrates three external LLM providers (Groq, Fireworks, Gemini), two databases (PostgreSQL and Redis), a background job queue (ARQ), and a semantic vector search engine (FAISS with Sentence Transformers). At the time of the audit, it exposes 14 REST API endpoints across three route groups: Chat, Sessions, and System Health.

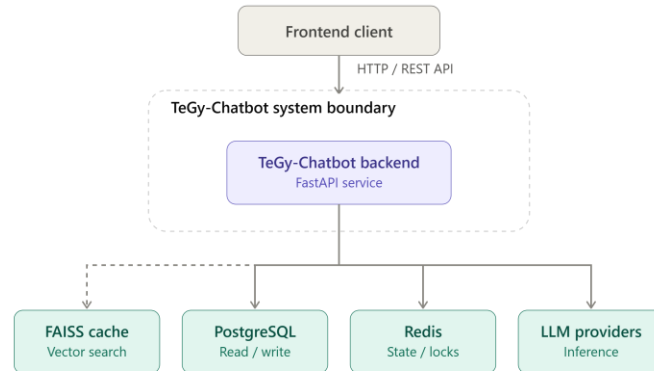
The codebase follows a strict separation of concerns: API controllers are thin delegators, business logic resides in the service layer, data access is abstracted through the repository pattern, and AI orchestration is encapsulated in a dedicated module with its own provider abstraction, intent routing, tool calling, and safety validation subsystems.

3.1 System Boundary

The system boundary defines how external actors interact with the core backend services. The following diagram illustrates the boundary crossing from the frontend client to external integrations:

3.2 Technology Stack

The following table summarizes the core technologies and frameworks utilized in the project:



Layer / Component	Technology
Backend Framework	FastAPI
ORM	SQLAlchemy 2.0
Relational Database	PostgreSQL
Cache & Distributed Locks	Redis
Background Queue	ARQ
Authentication	JWT (PyJWT)
AI Providers	Groq, Fireworks, Gemini
Embeddings	SentenceTransformers
Vector Search	FAISS
Containerization	Docker
Deployment	Azure Container Apps
CI/CD	GitHub Actions

4. Non-Functional Requirements

To ensure the system meets enterprise-grade operational standards, the architecture addresses several key Non-Functional Requirements (NFRs):

Requirement	Implementation & Strategy
Performance	Asynchronous I/O via FastAPI, semantic caching (FAISS) to bypass LLM latency, and a zero-LLM Fast Path for common intents.
Availability	Multi-provider LLM fallback strategy guarantees uninterrupted service even during upstream outages.
Reliability	Distributed Redis locks prevent race conditions, and ARQ handles background job processing with automatic retries.
Scalability	Stateless API design allows horizontal scaling via Azure Container Apps. Caching is handled centrally in Redis.
Security	JWT authentication, API key validation, input sanitization, parameterized SQL queries, and strict prompt injection firewalls.
Maintainability	Clean Architecture principles, dependency injection (IoC), and design patterns (Repository, Strategy, Factory) ensure code isolation.
Usability	Multi-language support (Arabic/English), fast response times, and conversational guided booking to reduce user friction.
Recoverability	Background retries, distributed locks, database persistence, and graceful degradation.

5. Business Value and Client Benefits

The following diagram illustrates the high-level business flow:



The following table presents the key features of TeGy-Chatbot from a business perspective, mapping each feature to its tangible benefit and the evidence of its implementation within the repository.

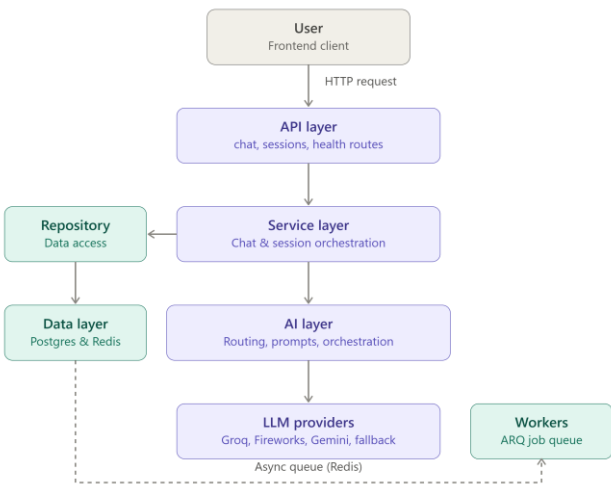
Feature	Business Benefit	Evidence
24/7 Automated Customer Support	Aims to reduce the need for human support agents by handling common queries (event info, booking status) autonomously.	app/ai/intent_detector.py: classifies intents including support_billing, support_technical
Multi-Provider LLM Failover	Ensures chatbot availability even when a single AI provider experiences downtime or rate limiting.	app/ai/providers/factory.py: PRIORITY_LIST = ["groq", "fireworks", "gemini"]
Zero-LLM Fast Path for Social Intents	Designed to reduce AI API costs by handling greetings, thanks, and goodbyes with zero LLM calls (estimated token reduction of approximately 1400 tokens according to internal configuration constants).	app/ai/fast_path_router.py:
Semantic Caching	Avoids redundant LLM calls for semantically similar questions, helping reduce latency and API costs.	app/services/semantic_cache_service.py
Automated Conversation Summarization	Designed to reduce token consumption for long conversations by summarizing older messages in the background.	app/workers/summarization_worker.py, triggered every 10 messages (app/ai/memory_manager.py)
Guided Booking via Tool Calling	Aims to reduce booking abandonment and human error by allowing the AI to execute database operations (search events, create tickets) on behalf of the user.	app/ai/tools/event_tools.py, app/ai/tools/ticket_tools.py
Idempotency and Distributed Locking	Prevents duplicate bookings and ensures data consistency under concurrent requests.	app/services/idempotency_service.py, app/infrastructure/adapters/redis_lock_adapter.py

6. System Architecture

The system follows an N-tier layered architecture with strict unidirectional data flow. The following diagram illustrates the major components and their interactions, with each box corresponding to a verified module in the codebase.

7. Message Flow and Request Lifecycle

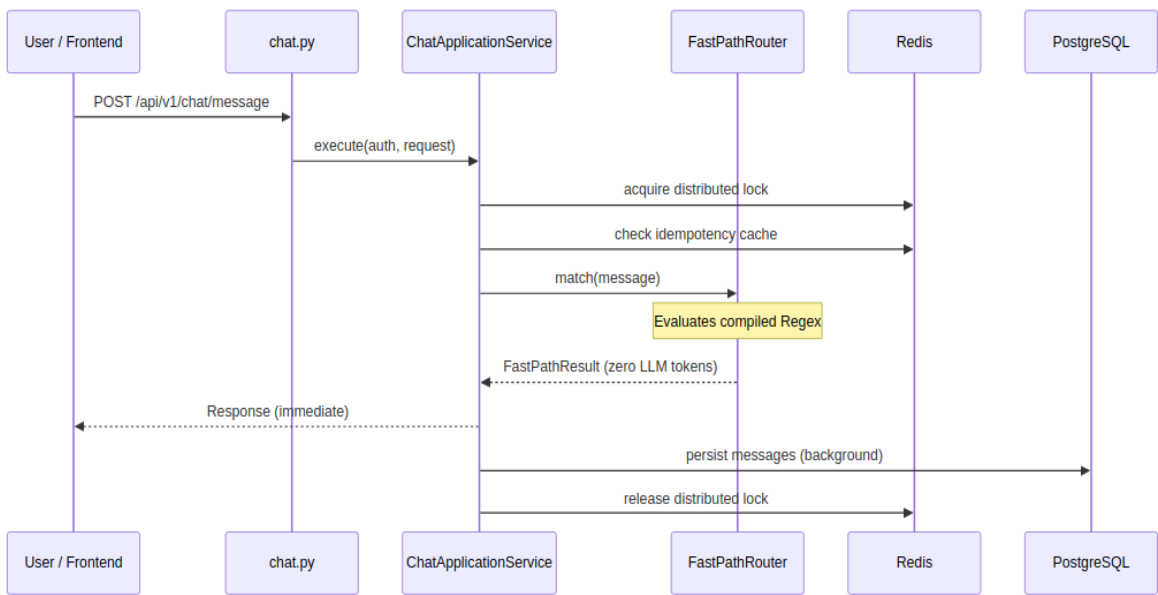
When a user sends a message via POST /api/v1/chat/message, the request traverses a multi-phase pipeline within ChatApplicationService.execute()



(app/services/chat_application_service.py). The lifecycle is divided into two main flows: the Fast Path (for social intents) and the LLM Pipeline (for complex requests).

7.1 Flow A: Fast Path (Social Intents)

The Fast Path intercepts simple intents like greetings and goodbyes, returning immediate branded responses without invoking the LLM.



7.2 Flow B: LLM Pipeline (Complex Requests)

If the message fails with the Fast Path regex evaluation, the system loads the session context and routes it through the semantic cache, intent detector, and the three-engine AI orchestrator.

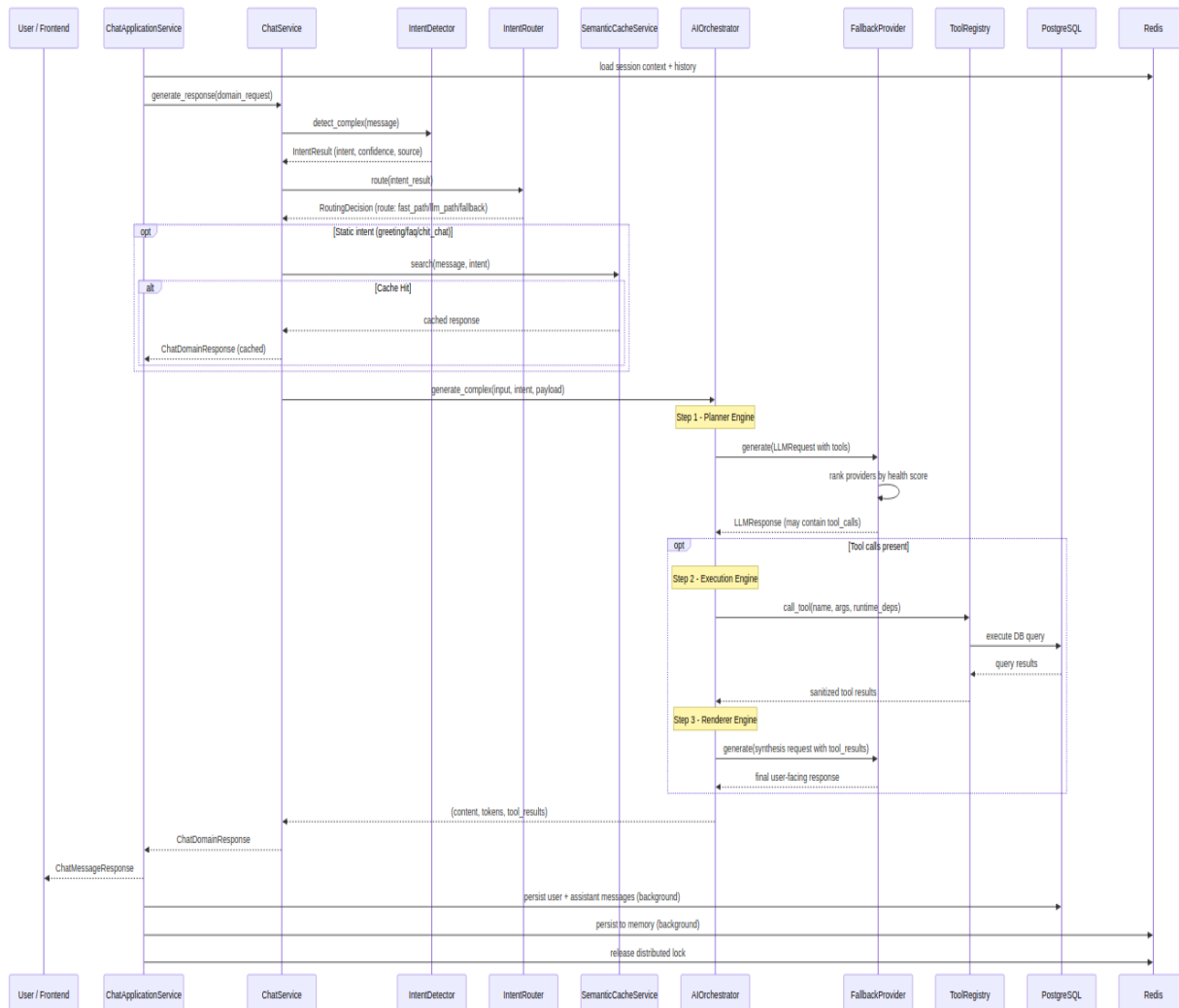
Pipeline Phases

The pipeline within `ChatApplicationService.execute()` operates in three sequential phases:

Phase 1 — Context Resolution: The system resolves the user identity from the JWT token, synchronizes the user profile, creates or retrieves the chat session, and loads conversation history from Redis.

Phase 2 — AI Generation: The `ChatService.generate_response()` method is invoked with a configurable timeout (`AI_REQUEST_TIMEOUT = 45` seconds, from `app/core/config.py`). This phase encompasses intent detection, routing, semantic cache lookup, and the three-engine orchestrator pipeline (Planner → Execution → Renderer).

Phase 3 — Persistence: User and assistant messages persist in PostgreSQL and Redis memory. On the success path, persistence is executed in a background task to minimize response latency, with the distributed lock transferred to the background task to guarantee strict message ordering.



8. AI and LLM Integration

8.1 Provider Architecture

The system integrates three LLM providers, each implemented as a subclass of the abstract LLMProvider base class (app/ai/providers/base.py). Providers are automatically discovered at startup via the ProviderRegistry (app/ai/providers/registry.py) and instantiated by the ProviderFactory (app/ai/providers/factory.py).

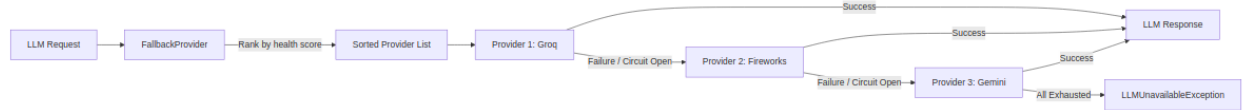
Provider	Model	Role	Fallback Order	Timeout	Source File
Groq	llama-3.3-70b-versatile	Primary (lowest latency)	1st	30s	app/ai/providers/groq_provider.py
Fireworks	qwen-v2p5-14b-instruct	Secondary fallback	2nd	30s	app/ai/providers/fireworks_provider.py
Gemini	gemini-3-flash-preview	Tertiary fallback	3rd	30s	app/ai/providers/gemini_provider.py

The fallback order is defined in app/ai/providers/factory.py:

```
# app/ai/providers/factory.py
PRIORITY_LIST = ["groq", "fireworks", "gemini"]
```

8.2 Fallback Mechanism

To ensure high availability and prevent single-point-of-failure scenarios when relying on external AI services, the system employs a robust fallback strategy. When multiple providers are configured, the factory wraps them in a FallbackProvider (app/ai/providers/fallback_provider.py). This provider implements health-based dynamic selection: providers are ranked by a real-time health score computed from their success rate (60% weight) and average latency (40% weight), modulated by circuit breaker state.



8.3 Resilience: Circuit Breaker and Retry

Network instability and API rate limits are common challenges when integrating third-party AI models. To protect the application from cascading failures and excessive latency, each provider is equipped with an independent circuit breaker and retry mechanism, both implemented in `app/ai/providers/resilience.py`.

- **Circuit Breaker** (`ProviderCircuitBreaker`): Transitions through three states — CLOSED → OPEN (after 5 consecutive failures) → HALF_OPEN (after 30-second cooldown) → CLOSED (on first success). These thresholds are configurable via `CIRCUIT_BREAKER_THRESHOLD` and `CIRCUIT_BREAKER_COOLDOWN_SECONDS` in `app/core/config.py`.
- **Retry with Exponential Backoff** (`retry_with_backoff`): Retries transient and timeout errors up to `LLM_MAX_RETRIES = 2` times with a base delay of 0.5 seconds (from `app/core/config.py`).

8.4 Intent Detection and Routing

Accurately understanding the user's goal is the first critical step in a conversational pipeline. Rather than routing all messages to an expensive LLM immediately, the intent detection system follows a hybrid approach implemented in `IntentDetector` (`app/ai/intent_detector.py`):

1. **Rule-Based Detection**: A fast, zero-cost first pass using keyword matching against predefined categories (billing, technical, event, booking, greeting). Each match yields a confidence score between 0.0 and 1.0.
2. **LLM Fallback**: When rule-based confidence falls below 0.6 or the intent is classified as general, the system falls back to the LLM for classification, costing tokens but providing higher accuracy.

The `IntentRouter` (`app/ai/intent_router.py`) translates the classified intent into a routing decision:

- **fast_path**: For greeting and booking intents with high confidence (≥ 0.8). Greeting goes directly to `FastPathRouter`; booking falls through to the full LLM pipeline.
- **llm_path**: Default route for most intents.
- **fallback_path**: For very low confidence results (< 0.5).

8.5 Three-Engine Orchestrator Pipeline

Handling complex multi-step tasks, such as searching for events and executing bookings, requires a structured reasoning and execution approach. To manage this safely and reliably, the `AIOrchestrator` (`app/services/ai_orchestrator.py`) implements a three-step pipeline for complex requests:

1. **Planner Engine:** Sends the user message to the LLM with available tools attached. The LLM decides whether to call tools or respond directly. If a tool hint is mapped to the intent (via INTENT_TOOL_MAP) and the LLM fails to call it, the planner retries with forced tool choice up to 2 times.
2. **Execution Engine:** Executes the tool calls returned by the planner via ToolRegistry.call_tool(), passing runtime database sessions as dependencies. Tool outputs are sanitized through a two-layer semantic firewall that strips forbidden keys (instruction, command, execute, etc.) and detects prompt injection patterns.
3. **Renderer Engine:** A "blind renderer" that receives only the tool results and conversation context—never the raw tool definitions—to synthesize a user-facing response. This separation prevents the renderer from hallucinating tool calls.

8.6 Fast Path Router

To minimize unnecessary LLM usage and guarantee instant responses for common interactions, the system introduces a lightweight routing layer. The FastPathRouter (app/ai/fast_path_router.py) intercepts social/meta intents (greeting, identity, thanks, goodbye) using compiled regex patterns and returns predefined branded responses in Arabic without any LLM calls. This optimization results in an estimated token reduction of approximately 1400 tokens according to internal configuration constants.

8.7 Tool Calling

The AI can invoke database-connected functions via the ToolRegistry (app/ai/tool_registry.py). Tools are auto discovered at startup by scanning app/ai/tools/ for modules with @ToolRegistry.register_tool decorators (app/ai/tools/__init__.py). The system currently registers three tools:

Tool Name	File	Purpose
search_events	app/ai/tools/event_tools.py	Searches for events using semantic search and database queries, returning relevant event details based on the user's request.
create_ticket	app/ai/tools/ticket_tools.py	Creates a ticket booking for a selected event after validating the required booking information.
search_orders	app/ai/tools/order_tools.py	Retrieves a user's previous ticket orders and booking history from the database.

The orchestrator dynamically filters which tools are available based on the detected intent (app/services/ai_orchestrator.py). For example, booking intents only receive 'check_availability', 'create_booking', and 'get_user_profile' tools.

8.8 Semantic Cache

To further help reduce latency and API costs for frequently asked questions, the system leverages vector-based caching for semantic similarity rather than exact text matching. The

SemanticCacheService (app/services/semantic_cache_service.py) provides an in-memory vector cache using FAISS (faiss-cpu==1.10.0) and SentenceTransformers (sentence-transformers==3.4.1 with the all-MiniLM-L6-v2 model, 384-dimensional embeddings). It is used exclusively for static intents (greeting, general_faq, chit_chat, fallback) as defined in app/services/chat_service.py. The cache features lazy loading to defer model initialization until the first request, and operates with a similarity threshold of 0.88, a TTL of 7 days, and a maximum capacity of 5,000 entries with FIFO eviction (from app/core/config.py, and app/services/semantic_cache_service.py).

9. Database Design

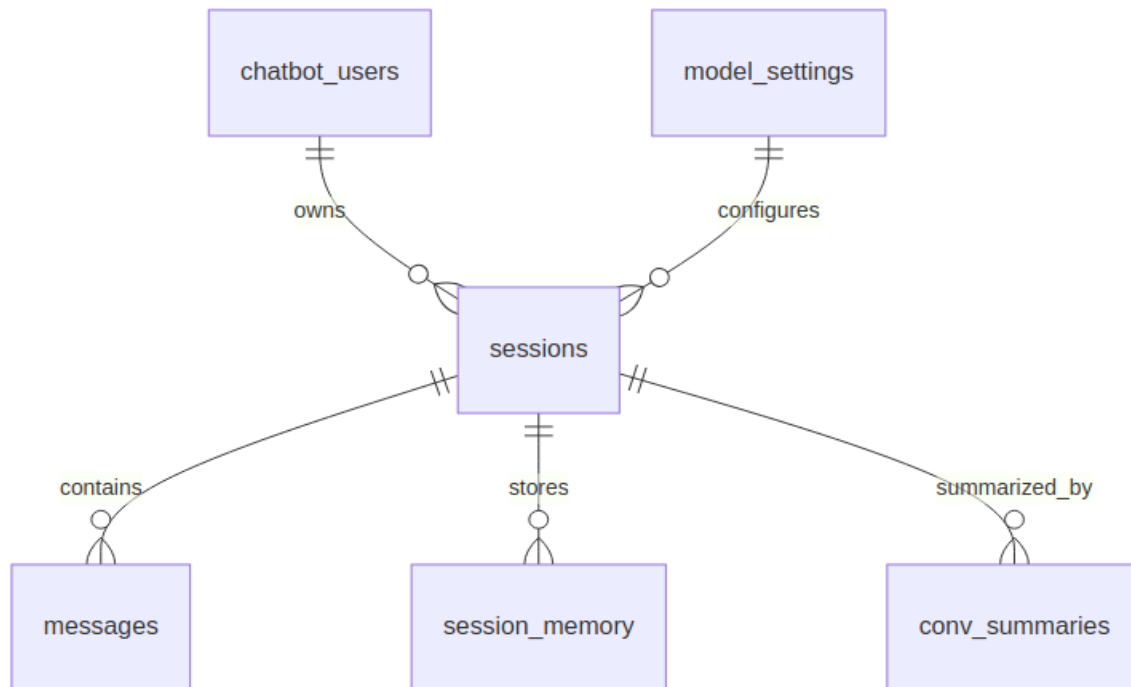
The system uses two separate PostgreSQL databases, both accessed asynchronously via SQLAlchemy 2.0 with the asyncpg driver.

- **Chatbot Database** (app/db/chatbot_database.py): Stores all chatbot-specific state—sessions, messages, user profiles, memory, summaries, and model settings.
- **Main Database** (app/db/main_database.py): The primary application database contains events, bookings, and user accounts. The chatbot reads from this database via tools but does not write to it.

Redis serves as a high-speed operational store for session context, recent message history (last 20 messages), message counters, distributed locks, and the ARQ job queue.

9.1 Entity-Relationship Diagram

The following ER diagram represents the chatbot database schema as defined by the SQLAlchemy ORM models in app/models/.



9.2 Database Normalization

The database schema adheres to Third Normal Form (3NF) to eliminate data redundancy and ensure data integrity. By separating the entities:

- **chatbot_users**: Stores core user identity independently of sessions.
- **sessions**: Represents conversational contexts without duplicating user details.
- **messages, session_memory, conv_summaries**: Each manages a specific aspect of the conversation, allowing the system to scale message storage, retrieve summaries, and manage memories without altering the core session record.

This separation of concerns ensures that the database structure remains optimal for high read/write operations characteristic of conversational interfaces.

9.3 Redis Data Schema

Redis keys follow a versioned naming convention managed by the RedisKeys class (app/db/redis.py):

Key Pattern	Data Type	TTL	Purpose
session: {id}: context	Hash	7 days	Stores session metadata, including user_id, current intent, conversation summary, and model information.
session: {id}: messages	List	24 hours	Stores the most recent 20 conversation messages using a trimmed LIFO list for context retrieval.

session: {id}:msg_count	String (Counter)	24 hours	Tracks the number of exchanged messages to determine when automatic conversation summarization should be triggered.
session: {id}: summarizing	String (Lock)	5 minutes	Acts as a distributed lock to ensure that only one summarization process runs for a session at a time.

These TTL values are defined in `app/ai/memory\_manager.py`.

10. Design Patterns

The following design patterns were identified and verified in the codebase:

Design Pattern	Evidence File(s)	Purpose
Repository Pattern	app/repositories/base_repo.py, session_repo.py, message_repo.py, chatbot_user_repo.py, summary_repo.py, memory_repo.py, model_settings_repo.py	Encapsulates all database operations behind reusable repository classes, separating data access logic from business logic and improving testability.
Factory Pattern	app/ai/providers/factory.py (ProviderFactory.initialize_provider_chain())	Centralizes the creation and configuration of LLM providers, building the provider chain based on application settings and priorities.
Strategy Pattern	app/ai/providers/base.py, groq_provider.py, fireworks_provider.py, gemini_provider.py	Provides interchangeable implementations of the LLMProvider interface, allowing runtime selection of the active LLM provider.
Chain of Responsibility	app/ai/providers/fallback_provider.py	Sequentially forwards requests through multiple LLM providers until one successfully processes the request or all providers fail.
Dependency Injection (IoC)	app/core/container.py (ServiceContainer)	Manages object creation and dependency resolution centrally, ensuring services receive their required dependencies without manual instantiation.

Decorator-Based Registration	app/ai/tool_registry.py, app/ai/providers/registry.py	Automatically registers AI tools and providers using decorators, enabling extensibility without modifying central registration code.
Adapter Pattern	app/infrastructure/adapters/redis_lock_adapter.py, redis_state_adapter.py, redis_resilience_adapter.py, ai_token_adapter.py	Adapts external infrastructure services such as Redis and token-counting utilities to interfaces expected by the application.
Template Method	app/services/ai_orchestrator.py	Defines a fixed execution workflow (Plan → Execute → Render) while allowing individual processing steps to be customized internally.
Facade Pattern	app/services/chat_application_service.py	Exposes a simplified interface that coordinates multiple services, repositories, and AI components to execute a complete chat request.
Builder Pattern	app/ai/prompt_builder.py	Incrementally constructs complex LLM prompts by combining system instructions, conversation history, memory, summaries, and tool context into a final prompt.

11. Background Job Processing

The system uses **ARQ** (arq==0.26.3), a Redis-based asynchronous task queue, for background job processing. The worker configuration is defined in app/workers/arq_jobs.py.

The primary background job is `summarize\_session\_job` (app/workers/arq_jobs.py), which delegates to `run\_summarization\_job` (app/workers/summarization_worker.py). This job is triggered when the message counter for a session reaches a multiple of 10 (SUMMARIZE_EVERY = 10, from app/ai/memory_manager.py). The summarization worker:

- Acquires a distributed lock to prevent duplicate summarization.

- Performs a Redis bucket-lock re-verification and a database-level idempotency guard.
- Fetches the last 30 messages from PostgreSQL (source of truth, not Redis).
- Calls the LLM to generate a summary via the `Summarizer` class.
- Persists the summary to the `conv\_summaries` table with a version number.
- Atomically decrements the message counter to preserve counts for messages that arrived during processing.

Failed jobs are retried with exponential backoff up to `ARQ\_MAX\_RETRIES = 3` times (`app/core/config.py`).

12. API Design

The API follows RESTful conventions with versioned routing under `/api/v1/`. All endpoints are defined in `app/api/v1/routes/` and registered in `app/main.py`. Authentication is handled via JWT tokens (PyJWT==2.12.1) and an API-key mechanism for integration of clients (`app/core/api\_key\_auth.py`).

12.1 Endpoint Reference

HTTP Method	API Endpoint	Purpose	Source File
POST	/api/v1/chat/message	Processes a user message, invokes the AI orchestration pipeline, and returns the generated response.	app/api/v1/routes/chat.py
POST	/api/v1/chat/session	Creates a new chat session and initializes the required session context.	app/api/v1/routes/chat.py
GET	/api/v1/chat/history/{session_id}	Retrieves the paginated conversation history for a specified chat session.	app/api/v1/routes/chat.py
GET	/api/v1/sessions	Returns a paginated list of chat sessions associated	app/api/v1/routes/sessions.py

		with the authenticated user.	
DELETE	/api/v1/sessions/{session_id}	Permanently deletes a chat session along with its associated conversation data.	app/api/v1/routes/sessions.py
PATCH	/api/v1/sessions/{session_id}	Updates session metadata, such as the session title or other editable attributes.	app/api/v1/routes/sessions.py
GET	/api/v1/health	Performs a comprehensive health check covering the database, Redis, background queue, and core services.	app/api/v1/routes/health.py
GET	/api/v1/health/cache/status	Returns cache usage statistics and operational metrics.	app/api/v1/routes/health.py
GET	/api/v1/health/cache/health	Verifies the operational status and availability of the caching layer.	app/api/v1/routes/health.py
POST	/api/v1/health/cache/clear/sessions	Clear all cached session-related data from Redis.	app/api/v1/routes/health.py
POST	/api/v1/health/cache/clear/prompts	Remove all cached prompt templates to	app/api/v1/routes/health.py

		force fresh loading.	
POST	/api/v1/health/cache/warm	Preload frequently used cache entries to improve application startup performance.	app/api/v1/routes/health.py
POST	/api/v1/health/cache/reset	Performs a complete cache reset by clearing and rebuilding cached data.	app/api/v1/routes/health.py
POST	/api/v1/health/cache/reload-prompt/{name}	Reloads a specific prompt template into the cache without restarting the application.	app/api/v1/routes/health.py

12.2 Payload Examples

The following JSON payloads demonstrate a typical interaction with the POST /api/v1/chat/message endpoint:

Request Example

```
{
  "user_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "session_id": "123e4567-e89b-12d3-a456-426614174000",
  "message": "Are there any tech conferences next week?",
  "role": "user"
  "user_profile": {
    "name": "string",
    "email": "string",
    "gender": "string"
  }
}
```

Response Example

```
{
  "session_id": "123e4567-e89b-12d3-a456-426614174000",
  "response": "Yes, I found two tech conferences happening next week in your area.",
  "is_new_user": true,
  "intent": "search_events",
  "token_usage": {
    "prompt_tokens": 120,
    "completion_tokens": 45,
    "total_tokens": 165
  }
}
```

12.3 Typical Request Flow

The following diagram illustrates the standard data flow from the client through the application layers to the database:



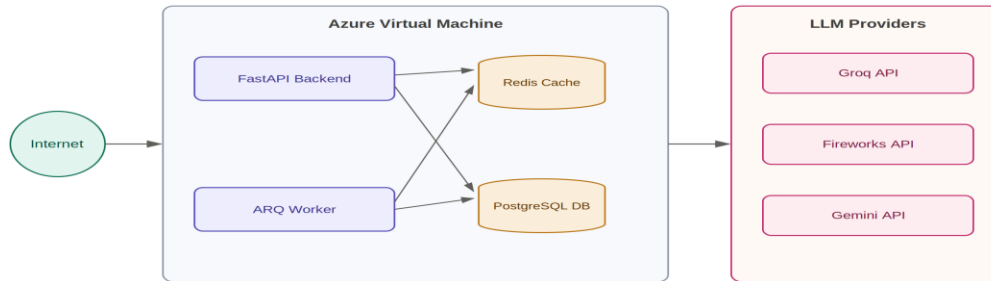
12.4 Request and Response Flow

API controllers are intentionally thin. The `send_chat_message` endpoint, for example, performs only authentication resolution before delegating entirely to the application service

The response format differs based on the authentication mode: `ChatIntegrationResponse` for API-key clients (containing only the response text) and `ChatMessageResponse` for JWT-authenticated users (containing session ID, metadata, and performance breakdown in debug mode).

13. Deployment and Infrastructure

The following diagram illustrates the deployment architecture and infrastructure dependencies, presenting a professional overview of the service topology:



13.1 Docker

The application is containerized using a Dockerfile based on python:3.11-slim. It installs system dependencies including Microsoft ODBC drivers (msodbcsql17) for SQL Server connectivity to the main application database, copies the project, and runs the application via Uvicorn on port 8000.

13.2 Docker Compose

The docker-compose.yml defines four services:

Service	Docker Image	Purpose
app	chatbot-app:latest (built from the project Dockerfile)	Hosts the main FastAPI application, exposing REST API endpoints and coordinating AI request processing.
arq-worker	chatbot-app:latest	Executes asynchronous background tasks using ARQ, including queued jobs that should not block API requests.
redis	redis:7-alpine	Provides in-memory caching, session management, distributed locking, and serves as the ARQ job queue backend.
postgres	postgres:15-alpine	Stores persistent application data, including users, chat sessions, messages, summaries, and related metadata during local development.

14. Security and Resilience

Enterprise conversational AI systems face unique vulnerabilities, ranging from traditional web exploits to LLM-specific prompt injection attacks. TeGy-Chatbot implements a defense-in-depth approach to secure user data and system integrity.

14.1 API and Authentication Security

- **JWT Authentication:** User sessions are strictly authenticated using JSON Web Tokens (JWT), ensuring data privacy and secure session tracking.

- **API Key Authentication:** Inter-service communication and integration clients utilize API keys (app/core/api_key_auth.py).
- **Secrets Management:** Sensitive credentials, database URIs, and LLM API keys are never hardcoded; they are injected securely via Environment Variables in Azure Container Apps.

14.2 Data and Database Security

- **Parameterized Queries:** SQLAlchemy 2.0 ORM is used exclusively for database interactions, inherently protecting the system against SQL Injection attacks.
- **Input Validation:** FastAPI's Pydantic models automatically validate and sanitize all incoming JSON payloads before they reach the service layer.

14.3 LLM and Prompt Security

The system implements a multi-layered defense against prompt injection attacks:

- **Input Layer:** InputSafetyGuard (app/ai/safety.py) blocks messages matching known injection patterns (e.g., 'ignore previous instructions', 'you are now').
- **History Layer:** ResponseValidator.sanitize_history() limits context sizes and drops history messages containing injection patterns to prevent context poisoning.
- **Tool Output Layer:** A semantic firewall in the orchestrator strips forbidden keys and injection markers from database tool results before passing them to the LLM.
- **Synthesis Isolation:** The Renderer Engine receives only tool results, never the raw tool definitions, completely preventing hallucinated tool execution.

14.4 Operational Resilience

- **Idempotency and Distributed Locks:** Redis-backed distributed locks (RedisLockAdapter) and idempotency keys guarantee that concurrent requests (e.g., rapid clicks on 'Book Ticket') do not result in duplicate transactions or race conditions.
- **Circuit Breakers and Retries:** Each LLM provider is guarded by an independent circuit breaker. Transient failures trigger exponential backoff retries, while sustained failures open the circuit and seamlessly fail over to the next provider.
- **Timeouts:** Strict timeouts (AI_REQUEST_TIMEOUT) prevent the system from hanging indefinitely if an upstream AI provider becomes unresponsive.

15. Future Work and Possible Extensions

1. **RAG and Vector Database:** Enhance the current semantic cache by integrating a dedicated Vector Database to support advanced Retrieval-Augmented Generation (RAG) capabilities for broader knowledge retrieval.
2. **Advanced Recommendation Engine:** Replace the current semantic search with a collaborative filtering system that learns from user booking history and preferences.

3. **Multi-Tenant Support:** Generalize the system to serve multiple event booking platforms with isolated data and custom branding.
4. **Automated Load Testing:** Implement performance benchmarking scripts to quantify response latencies, throughput under load, and token cost per conversation.
5. **Full Tool Implementation:** The `create_ticket` and `search_orders` tools currently return mock data (`app/ai/tools/ticket_tools.py`; `app/ai/tools/order_tools.py`). These should be connected to real booking and order management of APIs.
6. **Streaming Responses:** Implement Server-Sent Events (SSE) or WebSocket-based streaming to display AI responses token-by-token for improved perceived latency.
7. **Observability and OpenTelemetry:** Implement comprehensive observability using OpenTelemetry to trace requests across all microservices, LLM providers, and databases.
8. **Analytics Dashboard:** Develop a specialized dashboard to monitor chatbot performance, track user engagement, and visualize key metrics (e.g., token usage, fallback rates).
9. **Model Context Protocol (MCP):** Support additional external tools through MCP servers to expand the chatbot's operational scope.

16. Conclusion

TeGy-Chatbot represents a production-oriented implementation of an AI-powered conversational backend for the event ticketing domain. The system demonstrates several advanced software engineering practices: multi-provider LLM integration with health-based fallback, a three-engine orchestrator pipeline (Planner–Execution–Renderer) with semantic firewall protection, semantic vector caching using FAISS, distributed locking and idempotency for concurrent request safety, and automated conversation summarization via background workers.

The architecture prioritizes fault tolerance mechanisms and cost efficiency. The fast-path router eliminates LLM costs for social interactions, the semantic cache avoids redundant inference for repeated questions, and the summarization worker compresses conversation history to reduce token consumption. The system is fully containerized and deployed via a CI/CD pipeline to Azure Container Apps, demonstrating readiness for production workloads.

At the time of writing, the audited version of the project leverages a curated set of Python dependencies (from `requirements.txt`), integrates with 3 external LLM providers, manages 6 database tables across 2 PostgreSQL databases, and exposes 14 REST API endpoints—all organized within a clean, pattern-driven architecture that prioritizes testability and extensibility.

